CU7 Performance Assessment — LangganKu (starter)

Fill in your name and the last four digits of your IC, then work through the Tasks of the paper **in this notebook**. First create your own data file in a terminal: `python make_my_data.py <L4>` (the file `subscriber_churn.csv` appears in this folder). Keep the notebook runnable from top to bottom.

In [1]:
```
import warnings; warnings.filterwarnings("ignore")
import mlflow, numpy as np, pandas as pd

NAME = "CHIN KANG JIE"      # e.g. "AISYAH BINTI AHMAD"
L4   = "0873"                # last four digits of your IC, keep the leading zero

# ---- your personal values (rules in the paper) ----
d = [int(ch) for ch in L4]                    # e.g. "0685" -> [0, 6, 8, 5]
A1  = d[2] + 1
SEED      = int(L4)                           # random seed = L4 as a number
TEST_SIZE = 0.20 if d[3] % 2 == 0 else 0.25   # last digit even -> 0.20, odd -> 0.2
DEPTH     = 3 + d[1] % 4                      # RF max_depth = 3 + (2nd digit mod 4
C1 = round(0.04 * (d[2] + 1), 2)            # LogisticRegression C = 0.04 × (3rd IC
C2=10*C1
PORT      = 5100 + int(L4[2:])                # serving port = 5100 + last two digi
EXP = f"langganku_churn_{L4}"     # 实验名 → KeretaKu_Price_0873
REG = f"langganku_churn_{L4}"     # 注册模型名 → keretaku_price_0873
DB_URI = f"sqlite:///langganku_{L4}.db"
print(dict(SEED=SEED, TEST_SIZE=TEST_SIZE, DEPTH=DEPTH, C1=C1, C2=C2, PORT=PORT, EX
```

```
{'SEED': 873, 'TEST_SIZE': 0.25, 'DEPTH': 3, 'C1': 0.32, 'C2': 3.2, 'PORT': 5173, 'E
XP': 'langganku_churn_0873', 'REG': 'langganku_churn_0873', 'DB_URI': 'sqlite:///lan
gganku_0873.db'}
```

Task 1 — Set-up, split and pipeline

In [2]:
```
mlflow.set_tracking_uri(DB_URI)
mlflow.set_experiment(EXP)
```

Out[2]:
```
<Experiment: artifact_location='file:C:/Users/CHIN KANG JIE/Downloads/CHINKANGJIE_
CU7_PA/mlruns/1', creation_time=1792459393604, effective_trace_archival_retention=
None, experiment_id='1', last_update_time=1792459393604, lifecycle_stage='active',
name='langganku_churn_0873', tags={}, trace_location=None, workspace='default'>
```

In [3]:
```
df = pd.read_csv("subscriber_churn.csv")
print("shape:", df.shape)          # (1200, 10)
print(df.dtypes)
print("missing values:")
print(df.isna().sum())
```

```
shape: (1200, 10)
subscriber_id      object
avg_hours_week    float64
contract           object
support_calls       int64
plan               object
monthly_fee_rm    float64
payment_method     object
tenure_months       int64
main_device        object
churned             int64
dtype: object
missing values:
subscriber_id     0
avg_hours_week    0
contract          0
support_calls     0
plan              0
monthly_fee_rm    0
payment_method    0
tenure_months     0
main_device       0
churned           0
dtype: int64
```

In [4]:
```python
from sklearn.model_selection import train_test_split

X = df.drop(columns=["subscriber_id", "churned"])
y = df["churned"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=SEED
)
print(X_train.shape, X_test.shape)
```

```
(900, 8) (300, 8)
```

In [5]:
```python
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

# Identify categorical and numeric columns
cat_cols = ["contract", "plan", "payment_method", "main_device"]
num_cols = [c for c in X.columns if c not in cat_cols]

def make_pipeline(model):
    """Build a preprocessing + model pipeline, reused for every candidate."""
    preprocess = ColumnTransformer([
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
        ("num", "passthrough", num_cols),
    ])
    return Pipeline([("prep", preprocess), ("model", model)])
```

In [6]:
```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
def get_metrics(y_true, y_pred):
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1": f1_score(y_true, y_pred, zero_division=0),
    }

baseline = make_pipeline(LogisticRegression(max_iter=1000))
baseline.fit(X_train, y_train)

print("baseline test metrics:", get_metrics(y_test, baseline.predict(X_test)))
```

```
baseline test metrics: {'accuracy': 0.84, 'precision': 0.6976744186046512, 'recall':
0.46153846153846156, 'f1': 0.5555555555555556}
```

Task 2 — Track the candidates in ONE experiment

In [7]:
```
from sklearn.ensemble import RandomForestClassifier

def run_candidate(cand_name, model, model_type, settings):
    """Train the model inside a unified pipeline and log everything to MLflow."""
    # 1) Build and fit the pipeline
    pipe = make_pipeline(model)
    pipe.fit(X_train, y_train)

    # 2) Compute train and test metrics
    train_m = get_metrics(y_train, pipe.predict(X_train))
    test_m  = get_metrics(y_test,  pipe.predict(X_test))

    # 3) Log everything to MLflow in one run
    with mlflow.start_run(run_name=f"{cand_name}-{L4}"):
        mlflow.log_params({
            "model":      type(model).__name__,
            "test_size":  TEST_SIZE,
            "split_seed": SEED,
            **settings,               # model-specific hyper-params
        })
        mlflow.set_tag("model_type", model_type)
        mlflow.log_metrics({f"train_{k}": v for k, v in train_m.items()})
        mlflow.log_metrics({f"test_{k}": v for k, v in test_m.items()})
        mlflow.sklearn.log_model(
            pipe,
            name="model",
            input_example=X_train.head(5),
            serialization_format="cloudpickle",
        )

    print(f"{cand_name}-{L4}  test metrics: {test_m}")
    return test_m
```

In [8]:
```
# Candidate 1 — baseline LogisticRegression
run_candidate(
    "logreg-base",
    LogisticRegression(max_iter=1000),
```

```
    "linear",
    {},
)

# Candidate 2 — LogisticRegression with C = C1
run_candidate(
    f"logreg-c{C1:g}",
    LogisticRegression(C=C1, max_iter=1000),
    "linear",
    {"C": C1},
)
run_candidate(
    f"logreg-c{C2:g}",
    LogisticRegression(C=C2, max_iter=1000),
    "linear",
    {"C": C2},
)
# Candidate 3 — Random Forest
run_candidate(
    f"rf-d{DEPTH}",
    RandomForestClassifier(n_estimators=100, max_depth=DEPTH, random_state=SEED),
    "tree_ensemble",
    {"n_estimators": 100, "max_depth": DEPTH},
)
```

```
2026/10/20 12:17:36 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
logreg-base-0873  test metrics: {'accuracy': 0.84, 'precision': 0.6976744186046512, 'recall': 0.46153846153846156, 'f1': 0.5555555555555556}
2026/10/20 12:18:09 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
logreg-c0.32-0873  test metrics: {'accuracy': 0.83, 'precision': 0.6944444444444444, 'recall': 0.38461538461538464, 'f1': 0.49504950495049505}
2026/10/20 12:18:23 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
logreg-c3.2-0873  test metrics: {'accuracy': 0.8333333333333334, 'precision': 0.6666666666666666, 'recall': 0.46153846153846156, 'f1': 0.5454545454545454}
2026/10/20 12:18:35 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
```

```
rf-d3-0873  test metrics: {'accuracy': 0.81, 'precision': 1.0, 'recall': 0.123076923
07692308, 'f1': 0.2191780821917808}
```

Out[8]:
```
{'accuracy': 0.81,
 'precision': 1.0,
 'recall': 0.12307692307692308,
 'f1': 0.2191780821917808}
```

In [9]:
```
runs = mlflow.search_runs(experiment_names=[EXP])
print(len(runs), "runs")
print(runs[["tags.mlflow.runName", "tags.model_type",
            "metrics.train_f1", "metrics.test_f1"]])
```

```
44 runs
   tags.mlflow.runName tags.model_type  metrics.train_f1  metrics.test_f1
0           rf-d3-0873   tree_ensemble          0.237443         0.219178
1    logreg-c3.2-0873          linear          0.540373         0.545455
2   logreg-c0.32-0873          linear          0.536741         0.495050
3    logreg-base-0873          linear          0.540881         0.555556
4           rf-d3-0873   tree_ensemble          0.237443         0.219178
5    logreg-c3.2-0873          linear          0.540373         0.545455
6   logreg-c0.32-0873          linear          0.536741         0.495050
7    logreg-base-0873          linear          0.540881         0.555556
8           rf-d3-0873   tree_ensemble          0.237443         0.219178
9    logreg-c3.2-0873          linear          0.540373         0.545455
10  logreg-c0.32-0873          linear          0.536741         0.495050
11   logreg-base-0873          linear          0.540881         0.555556
12          rf-d3-0873   tree_ensemble          0.237443         0.219178
13   logreg-c3.2-0873          linear          0.540373         0.545455
14  logreg-c0.32-0873          linear          0.536741         0.495050
15   logreg-base-0873          linear          0.540881         0.555556
16          rf-d3-0873   tree_ensemble          0.237443         0.219178
17  logreg-c0.32-0873          linear          0.536741         0.495050
18   logreg-base-0873          linear          0.540881         0.555556
19  logreg-c0.32-0873          linear          0.536741         0.495050
20   logreg-base-0873          linear          0.540881         0.555556
21  logreg-c0.32-0873          linear          0.536741         0.495050
22   logreg-base-0873          linear          0.540881         0.555556
23  logreg-c0.32-0873          linear          0.536741         0.495050
24   logreg-base-0873          linear          0.540881         0.555556
25  logreg-c0.32-0873          linear          0.536741         0.495050
26   logreg-base-0873          linear          0.540881         0.555556
27  logreg-c0.32-0873          linear          0.536741         0.495050
28   logreg-base-0873          linear          0.540881         0.555556
29  logreg-c0.32-0873          linear          0.536741         0.495050
30   logreg-base-0873          linear          0.540881         0.555556
31  logreg-c0.32-0873          linear          0.536741         0.495050
32   logreg-base-0873          linear          0.540881         0.555556
33  logreg-c0.32-0873          linear          0.536741         0.495050
34   logreg-base-0873          linear          0.540881         0.555556
35  logreg-c0.32-0873          linear          0.536741         0.495050
36   logreg-base-0873          linear          0.540881         0.555556
37  logreg-c0.32-0873          linear          0.536741         0.495050
38   logreg-base-0873          linear          0.540881         0.555556
39          rf-d3-0873   tree_ensemble          0.237443         0.219178
40  logreg-c0.32-0873          linear          0.536741         0.495050
41   logreg-base-0873          linear          0.540881         0.555556
42  logreg-c0.32-0873          linear          0.536741         0.495050
43   logreg-base-0873          linear          0.540881         0.555556
```

Task 3 — Choose the best run from code and justify it

In [10]:
```
ranked = mlflow.search_runs(
    experiment_names=[EXP],
    order_by=["metrics.test_f1 DESC"],   # higher F1 is better
)
```

```
table = ranked[[
    "tags.mlflow.runName",
    "metrics.train_f1",
    "metrics.test_f1",
    "metrics.test_accuracy",
]].rename(columns={
    "tags.mlflow.runName": "run_name",
    "metrics.train_f1":    "train_f1",
    "metrics.test_f1":     "test_f1",
    "metrics.test_accuracy": "test_accuracy",
})
print(table.to_string(index=False))

best           = ranked.iloc[0]
best_run_id    = best["run_id"]
best_run_name  = best["tags.mlflow.runName"]
print("\nBest run:", best_run_name)
```

run_name	train_f1	test_f1	test_accuracy
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-base-0873	0.540881	0.555556	0.840000
logreg-c3.2-0873	0.540373	0.545455	0.833333
logreg-c3.2-0873	0.540373	0.545455	0.833333
logreg-c3.2-0873	0.540373	0.545455	0.833333
logreg-c3.2-0873	0.540373	0.545455	0.833333
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
logreg-c0.32-0873	0.536741	0.495050	0.830000
rf-d3-0873	0.237443	0.219178	0.810000
rf-d3-0873	0.237443	0.219178	0.810000
rf-d3-0873	0.237443	0.219178	0.810000
rf-d3-0873	0.237443	0.219178	0.810000
rf-d3-0873	0.237443	0.219178	0.810000
rf-d3-0873	0.237443	0.219178	0.810000

```
Best run: logreg-base-0873
```

Why I chose this run (2–3 sentences):

Task 4 — Register, alias and roll back

In [11]:
```
from mlflow import MlflowClient
client = MlflowClient()
```

```
# Find the run_id of logreg-base (version 1 = baseline)
base_run_id = ranked.loc[
    ranked["tags.mlflow.runName"] == f"logreg-base-{L4}", "run_id"
].iloc[0]

# Register as version 1 and give it the alias 'champion'
mv1 = mlflow.register_model(f"runs:/{base_run_id}/model", REG)
client.set_registered_model_alias(REG, "champion", mv1.version)

print("version", mv1.version, "-> champion")
```

```
Registered model 'langganku_churn_0873' already exists. Creating a new version of th
is model...
2026/10/20 12:18:49 WARNING mlflow.tracking._model_registry.fluent: Run with id 15ba
d0575d954537a7d42bb194d4e857 has no artifacts at artifact path 'model', registering
model based on models:/m-bbc1e8bfbe124ff3b9102b42e3568e52 instead
version 9 -> champion
Created version '9' of model 'langganku_churn_0873'.
```

In [12]:
```
# Register the best run (from Task 3) as version 2 and move champion
mv2 = mlflow.register_model(f"runs:/{best_run_id}/model", REG)
client.set_registered_model_alias(REG, "champion", mv2.version)

print("version", mv2.version, "-> champion")
print("champion now points to:", client.get_model_version_by_alias(REG, "champion")
```

```
Registered model 'langganku_churn_0873' already exists. Creating a new version of th
is model...
2026/10/20 12:18:49 WARNING mlflow.tracking._model_registry.fluent: Run with id 15ba
d0575d954537a7d42bb194d4e857 has no artifacts at artifact path 'model', registering
model based on models:/m-bbc1e8bfbe124ff3b9102b42e3568e52 instead
version 10 -> champion
champion now points to: 10
Created version '10' of model 'langganku_churn_0873'.
```

In [13]:
```
# Add tag and description to version 2
client.set_model_version_tag(REG, "2", "approved_by", NAME)
client.update_model_version(
    REG, "2",
    description="Logistic Regression with C=0.32 chosen by highest test_f1 for Pinj
)
```

Out[13]:
```
<ModelVersion: aliases=[], creation_timestamp=1792466892127, current_stage='None',
deployment_job_state=None, description=('Logistic Regression with C=0.32 chosen by
highest test_f1 for PinjamPantas '
 'loan default prediction.'), last_updated_timestamp=1792469929814, metrics=None,
model_id=None, name='langganku_churn_0873', params=None, run_id='2ad8e91a86d444f79
08ad9c0babaae0d', run_link=None, source='models:/m-b60bd51100d2422c954ceb5ab6b1b6d
9', status='READY', status_message=None, tags={'approved_by': 'CHIN KANG JIE'}, us
er_id=None, version=2, workspace='default'>
```

In [14]:
```
# Roll-back: champion -> version 1
client.set_registered_model_alias(REG, "champion", "1")
print("after roll-back   :", client.get_model_version_by_alias(REG, "champion").ver
```

```
# Roll-forward: champion -> version 2
client.set_registered_model_alias(REG, "champion", "2")
print("after roll-forward:", client.get_model_version_by_alias(REG, "champion").ver
```

```
after roll-back   : 1
after roll-forward: 2
```

Task 5 — Serve the champion and call it

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: